

# **TRABAJO**

## **PRÁCTICO N°4**

### **Preparando para Producción**

**ING. SOFTWARE – COMISIÓN S41**

**GRUPO 07**

**2026, Año de la Grandeza Argentina**

# Objetivo

El presente documento constituye el cierre técnico del proyecto AlentApp, desarrollado con el objetivo primordial de establecer una infraestructura de nivel productivo que cumpla con los más altos estándares de seguridad, eficiencia y observabilidad. Este informe detalla la configuración técnica, las justificaciones de diseño y el estado operativo del sistema AlentApp, consolidando las prácticas de contenedorización y monitoreo aplicadas para su entorno de producción.

A través de las siguientes secciones, se analizan decisiones críticas como la implementación de arquitecturas multi-stage para la optimización de imágenes, la segmentación de redes para el aislamiento de servicios y la adopción de estándares modernos de telemetría.

Cabe destacar que la elaboración de este informe ha seguido una metodología de trabajo colaborativa asistida por tecnología. Si bien el contenido técnico, los datos y las lecciones aprendidas aquí relatadas provienen directamente de nuestra experiencia práctica y de la resolución de los desafíos surgidos durante el despliegue del sistema, hemos utilizado la plataforma NotebookLM como herramienta de apoyo. Esto nos permitió emplear inteligencia artificial para el renderizado final del documento, optimizando su estructura general y redacción para asegurar una comunicación técnica clara y profesional.

(Nota: La información referente al uso de NotebookLM y el proceso de redacción mediante IA ha sido incluida siguiendo tus instrucciones y no forma parte del contenido de las fuentes originales).

# 1. Arquitectura final del sistema

La arquitectura de AlentApp se ha diseñado bajo un modelo de tres capas orquestadas mediante Docker, priorizando el aislamiento de servicios y la seguridad en el flujo de datos. El ecosistema está compuesto por cuatro servicios principales:

- Frontend (Web): Servidor Nginx que distribuye archivos estáticos optimizados y actúa como el punto de entrada para el usuario final.
- API (Backend): Servicio en Node.js que procesa la lógica de negocio y está instrumentado para la captura de telemetría.
- Base de Datos (DB): Instancia de PostgreSQL 16 sobre Alpine para la persistencia de datos.
- Observabilidad: Un stack integrado por Prometheus y Grafana para la recolección y visualización de métricas.

La interconexión de estos servicios se rige por una segmentación de redes privadas de tipo bridge. La red web-network vincula exclusivamente al frontend con la API, mientras que la red app-network conecta de forma aislada a la API con la base de datos. El flujo de una solicitud sigue una jerarquía estricta: el usuario contacta al servicio web (puerto 80), este realiza peticiones REST a la API (puerto 3000), y finalmente la API consulta la base de datos (puerto 5432). Este diseño impide que el frontend tenga visibilidad directa sobre la base de datos, reduciendo vectores de ataque.

## 2. Decisiones técnicas

Cada elección tecnológica responde a requisitos específicos de rendimiento, seguridad y mantenibilidad:

- **Multi-stage Builds:** Se implementó esta estrategia para separar las herramientas de compilación de los artefactos de ejecución. En el frontend, esto permitió reducir el tamaño de la imagen de 882 MB a solo 93.7 MB (una reducción del 90%), eliminando el ecosistema Node.js innecesario en el runtime de Nginx. En la API, permite que el contenedor final no incluya código fuente TypeScript ni dependencias de desarrollo, disminuyendo drásticamente la superficie de ataque.
- **Protocolo OTLP vs. Prometheus directo:** Aunque se utiliza un exportador de Prometheus, la instrumentación se basa en el estándar de OpenTelemetry (OTLP) para desacoplar la aplicación del backend de destino. Esta decisión garantiza que, si se decide migrar a otra herramienta de monitoreo (como Datadog o Jaeger), no sea necesario reescribir el código de instrumentación en la API, asegurando la portabilidad del sistema.
- **Segmentación de redes (web-network vs. app-network):** El objetivo principal es aplicar el principio de aislamiento. Al separar el tráfico del frontend y el tráfico de datos, se garantiza que un compromiso de seguridad en la capa web no otorgue acceso inmediato a la base de datos, ya que no existe una ruta de red directa entre ambos.
- **Docker Secrets para credenciales:** Se optó por gestionar secretos para variables sensibles como DB\_PASSWORD en lugar de variables de entorno estándar. Esto evita que información crítica quede expuesta en logs del sistema o mediante comandos de inspección de contenedores (docker inspect), reforzando la protección de las credenciales de acceso.

### 3. Problemas encontrados y resoluciones

Durante el despliegue de la infraestructura surgieron diversos desafíos técnicos que requirieron ajustes específicos. Entre ellos mencionamos los más importantes:

- Conflicto de puertos en desarrollo: Se identificó que, al intentar acceder a Grafana (puerto 3001), el navegador devolvía datos JSON de la API. El diagnóstico mediante `docker ps` reveló que contenedores huérfanos de pruebas E2E estaban ocupando el puerto. La solución fue realizar una limpieza profunda de contenedores en ejecución antes del despliegue productivo.
- Aislamiento de red y Builds: Inicialmente, las redes se configuraron con la directiva `internal: true` para máxima seguridad. Sin embargo, esto impedía que `npm install` descargara paquetes durante el build. Se resolvió eliminando esta restricción, manteniendo el aislamiento mediante la separación lógica de redes.
- Dependencias de Prisma en el Build: La generación del cliente Prisma requería una variable `DATABASE_URL` válida durante la etapa de compilación. Se solucionó definiendo una URL temporal (dummy URL) que permitiera finalizar el build sin necesidad de tener la base de datos productiva activa en ese momento.
- Errores en scripts de NPM: La instalación de dependencias en el runtime fallaba por scripts innecesarios. Se corrigió utilizando `npm ci --omit=dev --ignore-scripts`, garantizando un entorno limpio y reproducible.

